

NLP: Delivery 1 Report

Baseline Model

To define the foundations of the results we can obtain in the Quora challenge, we have implemented a basic solution, before proceeding to more complex architectures. Our baseline model consists of a Logistic Regression classifier combined with a *CountVectorizer*. This first approach allows us to set a performance benchmark and to understand the nature of the data.

In the development of this model, as part of the experimenting with the results, we tried to lemmatize the questions before applying the *CountVectorizer*, however this step degraded the performance noticeably. We believe this may be a cause of the fact that verb tenses and exact grammatical forms often dictate the specific intent of a question (example: "What happened?" versus "What will happen?"). Therefore, by collapsing these words we might have been removing important information.

Furthermore, we observed an interesting dynamic regarding vectorization techniques when improving our simple solution. We deliberately chose to maintain the *CountVectorizer* to build our main feature space because it empirically yielded better results than *TfidfVectorizer*. However, we modified the simple model approach. Instead of simply stacking the independent transformations of both questions, we subtract and multiply the resulting matrices of the *BoW* algorithm and we stack the results. Like this, we expect not to give separate information of the two questions, but to give explicit relational and comparative values. Conversely, we specifically utilized *TfidfVectorizer* within our handcrafted features to compute the cosine similarity distance. For this metric, *TF-IDF* is superior because it effectively highlights rare, topic-defining words while giving less importance to common stop words.

- What problems/limitations do you think the model has?

Of course, our baseline model will suffer the effects of the *BoW* approach, which is the lack of contextual understanding, the inability to identify synonyms and the loss of sequential information. The problem of equal token weighting is also present, but in this case is mitigated by the use of *TF-IDF* in the handcrafted features.

- What type of errors do you get?

Due to the limitations that we have just described, we can find in our mistakes several false positives caused by questions that have a lot of common words but have different objectives. The following case is an example:

```
How should I learn to code?  
Should I learn how to code?  
true class: 0  
prediction: 1
```

Also, we can find several false negatives in questions that use synonyms or different grammatical structures to ask the same thing. The following is an example:

```
What is a good solar panel installation provider near Aptos, California CA?  
What is a suitable solar panel installation provider near Aptos, California CA?  
true class: 1  
prediction: 0
```

- **What type of features can you build to improve the basic naive solution?**

To improve this naive model, we have implemented a set of features that try to capture some structural and pattern-based similarities that are not considered at the moment.

To start, we have calculated some distance and overlap metrics like Jaccard similarity, Common-word F1 ratio, the Character-bigram Dice coefficient and the *TF-IDF* Cosine Similarity. These metrics allow us to measure token intersection and capture similarity weighted by term rareness.

Moreover, we introduced a length ratio metric to penalize question pairs with a big difference in the size, and we added other features like exact number matching, uppercase entity matching and a feature checking if both questions start with the exact same word (to distinguish between "How" and "Why").

With these new features, we have been able to increase the training and validation accuracy by a 1%.

Improved Model

To further improve the previous solution, we moved from purely lexical overlap features to semantic representations. In particular, we encoded each question independently with paraphrase-MiniLM-L6-v2, a Sentence-BERT model specialised in paraphrase identification. Once both embeddings are obtained, we do not simply concatenate them. Instead, we again construct interaction features by combining the element-wise difference and the element-wise product of both vectors. This decision follows the same intuition as in the baseline: the model should not only receive separate information about q1 and q2, but also an explicit representation of how both questions relate to each other.

However, semantic information alone was not enough to obtain the best results. For this reason, we extended the model with graph-based features extracted from the question-pair graph. In this graph, each unique question is represented as a node and each pair of questions in the dataset defines an edge. From this structure, we compute how often each question appears in the training data, the minimum and maximum of both frequencies, the absolute frequency difference and the size of the common-neighbour set. The intuition is that if two questions share neighbours in the graph, they are more likely to be duplicates, because they participate in similar paraphrase relations across the dataset.

An important design decision is that all resources used by the improved model are built from the training split only. This includes the TF-IDF statistics used in the cosine feature and the graph dictionaries used at evaluation time. By doing this, we avoid leaking information from validation or test into the feature computation and we ensure that the reported results are fair and reproducible.

Why does the improved model work better?

The baseline model is strong when duplicate questions share many words, but it struggles when two questions express the same meaning using different vocabulary. The semantic encoder directly addresses this issue by placing paraphrases closer in embedding space. In parallel, the graph features add structural information that is not present in either the bag-of-words representation or the dense embeddings. In practice, this combination turns out to be the most effective, because it joins lexical evidence, semantic similarity and dataset-level relational patterns in the same classifier.

Implemented features and code contribution

Inside `utils.py`, the improved pipeline contains several features implemented from scratch. These include length ratio, common-word F1, character-bigram Dice coefficient, number mismatch, exact number match, uppercase entity matching, first-word matching and TF-IDF cosine similarity computed with sparse operations. In addition,

a word-level Longest Common Subsequence ratio was also implemented and demonstrated separately, although it was not included in the final full pipeline because of its computational cost.

For the semantic model, the function `get_sbert_interaction_features` transforms each question pair into a dense interaction vector based on Sentence-BERT embeddings. For the graph-enhanced system, `build_freq_dict` and `build_neighbor_dict` create the dictionaries needed to represent the training question graph, while `get_graph_features` and `get_sbert_graph_features` combine this structural information with the semantic representation used by the classifier. This organisation keeps the notebooks clean and follows the project guideline of placing reusable logic in external utility code.

Results

The improved systems clearly outperform the simple semantic-only alternative and also improve over the baseline in ROC-AUC. The best model is the SBERT + Graph system. Its validation ROC-AUC is 0.9448 and its test ROC-AUC is 0.9443. By comparison, the baseline model reaches 0.9007 on validation and 0.9061 on test. This shows that the graph signal contributes a substantial gain and generalises well across splits.

What problems/limitations do you think the improved model has?

Even though the improved model performs much better, it still has several limitations. First, the Sentence-BERT encoder is more computationally expensive than the bag-of-words approach, both in memory and in processing time. Second, the graph-based features depend on patterns that are visible in the training graph. If a question is rare or completely unseen, these features become much less informative. Third, the final classifier is still a relatively simple linear model on top of fixed features, so it cannot model very fine-grained interactions such as subtle negation, pragmatic intent or complex compositional reasoning as well as a cross-encoder could.

In addition, the model can still be sensitive to numerical details, named entities and hidden world knowledge. Two questions may be topically similar and therefore close in embedding space, but still differ in a crucial condition such as time, quantity or location. In those cases, the system may assign a high similarity score even when the questions are not exact duplicates.

Work distribution

- Álvaro Soto was responsible for data loading, exploratory analysis and the full baseline pipeline, including the vectorisers, interaction features and error inspection utilities. Also write the Baseline part model in this file.
- Arnau Soler implemented the improved model, including handcrafted similarity features, SBERT interaction features, graph-based features and the reproduce_results notebook. Also write the Improved part model in this file.
- Malik Sattar contributed to the project in a general way across the different stages of the work.

Reproducibility notes

Following the deliverable guidelines, the project is organised so that train_models.ipynb stores the trained artifacts inside a models folder and reproduce_results.ipynb loads them to compute train, validation and test metrics without retraining. The data is read from ~/Datasets/QuoraQuestionPairs/quora_data.csv and the environment is described in environment.yml so that the project can be executed in another machine with the same setup.